MONGODB BASIC COMMANDS

==========

1. CREATE / USE DATABASE

---

use collegeDB

Output:

switched to db collegeDB

Show current database:

db

Output:

collegeDB

---

2. CREATE COLLECTION

---

db.createCollection("students")

Output:

{ ok: 1 }

Show collections:

show collections

Output:

students

---

3. INSERT ONE DOCUMENT

```
db.students.insertOne({
  name: "Radhika Sharma",
  age: 20,
  city: "Pune",
  course: "B.Tech IT",
  marks: 82
})
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId("...")
}
```

---

4. INSERT MANY DOCUMENTS

```
db.students.insertMany([
  {
    name: "Priya Patil",
    age: 21,
    city: "Nashik",
```

```
  course: "B.Tech IT",
  marks: 91
},
{
  name: "Rahul Deshmukh",
  age: 20,
  city: "Mumbai",
  course: "B.Tech CS",
  marks: 76
},
{
  name: "Sneha Kulkarni",
  age: 22,
  city: "Nagpur",
  course: "BCA",
  marks: 88
},
{
  name: "Rohan Joshi",
  age: 19,
  city: "Aurangabad",
  course: "B.Tech IT",
  marks: 69
},
{
  name: "Neha Pawar",
  age: 21,
  city: "Nashik",
  course: "B.Sc CS",
  marks: 95
}
```

```
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    "0": ObjectId("..."),
    "1": ObjectId("..."),
    "2": ObjectId("..."),
    "3": ObjectId("..."),
    "4": ObjectId("...")
  }
}
```

---

5. FIND ALL DOCUMENTS

```
db.students.find()
```

Output:

```
[
  {
    _id: ObjectId("..."),
    name: "Aarav Sharma",
    age: 20,
    city: "Pune",
    course: "B.Tech IT",
    marks: 82
  },
```

...

]

---

6. FIND ONE DOCUMENT

```
db.students.findOne({ name: "Priya Patil" })
```

Output:

```
{
  _id: ObjectId("..."),
  name: "Priya Patil",
  age: 21,
  city: "Nashik",
  course: "B.Tech IT",
  marks: 91
}
```

---

7. FIND WITH EQUALITY FILTER

```
db.students.find({ city: "Nashik" })
```

Output:

```
[
  { name: "Priya Patil", city: "Nashik", marks: 91, ... },
  { name: "Neha Pawar", city: "Nashik", marks: 95, ... }
```

]

---

8. FIND WITH GREATER THAN ($gt)

---

db.students.find({ marks: { $gt: 85 } })

Output:

[

{ name: "Priya Patil", marks: 91, ... },

{ name: "Sneha Kulkarni", marks: 88, ... },

{ name: "Neha Pawar", marks: 95, ... }

]

---

9. FIND WITH GREATER THAN OR EQUAL ($gte)

---

db.students.find({ marks: { $gte: 90 } })

Output:

[

{ name: "Priya Patil", marks: 91, ... },

{ name: "Neha Pawar", marks: 95, ... }

]

---

10. FIND WITH LESS THAN ($lt)

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
[
  { name: "Rahul Deshmukh", marks: 76, ... },
  { name: "Rohan Joshi", marks: 69, ... }
]
```

---

11. FIND WITH LESS THAN OR EQUAL ($lte)

```
db.students.find({ age: { $lte: 20 } })
```

Output:

```
[
  { name: "Aarav Sharma", age: 20, ... },
  { name: "Rahul Deshmukh", age: 20, ... },
  { name: "Rohan Joshi", age: 19, ... }
]
```

---

12. NOT EQUAL ($ne)

```
db.students.find({ city: { $ne: "Nashik" } })
```

Output:

All students whose city is not Nashik.

---

13. IN OPERATOR ($in)

```
db.students.find({
  city: { $in: ["Nashik", "Pune", "Mumbai"] }
})
```

Output:

Students from Nashik, Pune, or Mumbai.

---

14. NOT IN OPERATOR ($nin)

```
db.students.find({
  city: { $nin: ["Nashik", "Pune"] }
})
```

Output:

Students whose city is neither Nashik nor Pune.

---

15. AND CONDITION

```
db.students.find({
  course: "B.Tech IT",
  marks: { $gt: 80 }
})
```

Output:

```
[
  { name: "Aarav Sharma", course: "B.Tech IT", marks: 82, ... },
  { name: "Priya Patil", course: "B.Tech IT", marks: 91, ... }
]
```

---

16. OR CONDITION ($or)

```
db.students.find({
  $or: [
    { city: "Nashik" },
    { city: "Pune" }
  ]
})
```

Output:

Students from Nashik or Pune.

---

17. NOR CONDITION ($nor)

```
db.students.find({
  $nor: [
    { city: "Nashik" },
    { marks: { $gt: 90 } }
  ]
})
```

Output:

Students who are not from Nashik AND do not have marks above 90.

---

18. EXISTS OPERATOR ($exists)

```
db.students.find({
  marks: { $exists: true }
})
```

Output:

All documents containing the marks field.

---

19. FIND WITH MULTIPLE CONDITIONS

```
db.students.find({
  age: { $gte: 20, $lte: 21 },
```

```
  marks: { $gte: 80 }
})
```

Output:

Students aged 20–21 with marks 80 or above.

---

20. PROJECTION — SHOW ONLY SELECTED FIELDS

```
db.students.find(
  {},
  { name: 1, marks: 1 }
)
```

Output:

```
[
  { name: "Aarav Sharma", marks: 82 },
  { name: "Priya Patil", marks: 91 },
  ...
]
```

---

21. PROJECTION — HIDE A FIELD

```
db.students.find(
  {},
  { _id: 0, name: 1, city: 1 }
```

)

Output:

[

{ name: "Aarav Sharma", city: "Pune" },

{ name: "Priya Patil", city: "Nashik" },

...

]

---

22. SORT ASCENDING

---

db.students.find().sort({ marks: 1 })

Output:

Students arranged from lowest marks to highest marks.

---

23. SORT DESCENDING

---

db.students.find().sort({ marks: -1 })

Output:

Students arranged from highest marks to lowest marks.

---

24. SORT BY MULTIPLE FIELDS

```
db.students.find().sort({
  city: 1,
  marks: -1
})
```

Output:

Sorted by city ascending and marks descending within each city.

---

25. LIMIT

```
db.students.find().limit(3)
```

Output:

First 3 documents.

---

26. SKIP

```
db.students.find().skip(2)
```

Output:

All documents except the first 2.

---

27. SKIP + LIMIT

```
db.students.find()
  .skip(2)
  .limit(2)
```

Output:

2 documents after skipping the first 2.

---

28. COUNT DOCUMENTS

```
db.students.countDocuments()
```

Output:

7

Note:

The exact number depends on how many documents you inserted.

---

29. COUNT WITH FILTER

```
db.students.countDocuments({
```

```
  marks: { $gte: 80 }
})
```

Output:

Number of students having marks >= 80.

---

30. UPDATE ONE — $set

```
db.students.updateOne(
  { name: "Aarav Sharma" },
  { $set: { marks: 85 } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

---

31. UPDATE MANY — $set

```
db.students.updateMany(
  { city: "Nashik" },
  { $set: { state: "Maharashtra" } }
```

)

Output:

{

acknowledged: true,

matchedCount: 2,

modifiedCount: 2

}

---

32. INCREMENT VALUE — $inc

---

db.students.updateOne(

{ name: "Rahul Deshmukh" },

{ $inc: { marks: 5 } }

)

Output:

{

acknowledged: true,

matchedCount: 1,

modifiedCount: 1

}

Rahul's marks increase by 5.

---

33. DECREMENT VALUE — $inc

```
db.students.updateOne(
  { name: "Rohan Joshi" },
  { $inc: { marks: -2 } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

---

34. RENAME FIELD — $rename

```
db.students.updateMany(
  {},
  { $rename: { course: "program" } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 7,
  modifiedCount: 7
}
```

Note:

Run this only once because the field course is renamed to program.

---

35. ADD ARRAY FIELD — $push

```
db.students.updateOne(
  { name: "Priya Patil" },
  { $push: { skills: "MongoDB" } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

---

36. ADD MULTIPLE ARRAY VALUES — $push + $each

```
db.students.updateOne(
  { name: "Priya Patil" },
  {
    $push: {
      skills: {
        $each: ["Java", "React"]
```

```
    }
   }
  }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

---

37. REMOVE ARRAY VALUE — $pull

```
db.students.updateOne(
  { name: "Priya Patil" },
  { $pull: { skills: "Java" } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

---

38. UNSET / REMOVE A FIELD — $unset

```
db.students.updateOne(
  { name: "Priya Patil" },
  { $unset: { state: "" } }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

---

39. REPLACE ONE DOCUMENT

```
db.students.replaceOne(
  { name: "Rohan Joshi" },
  {
    name: "Rohan Joshi",
    age: 20,
    city: "Aurangabad",
    program: "B.Tech IT",
    marks: 75
  }
)
```

Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

Note:

replaceOne replaces the complete document except _id.

---

40. DELETE ONE

```
db.students.deleteOne({
  name: "Rohan Joshi"
})
```

Output:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

---

41. DELETE MANY

```
db.students.deleteMany({
```

```
  marks: { $lt: 70 }
})
```

Output:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

---

42. DELETE ALL DOCUMENTS

```
db.students.deleteMany({})
```

Output:

```
{
  acknowledged: true,
  deletedCount: ...
}
```

WARNING:

This deletes every document from students.

---

43. DROP COLLECTION

```
db.students.drop()
```

Output:

true

WARNING:

This deletes the complete students collection.

---

44. DROP DATABASE

---

db.dropDatabase()

Output:

{ ok: 1, dropped: "collegeDB" }

WARNING:

This deletes the complete current database.

==========

QUICK REVISION

==========

CREATE DATABASE:

use collegeDB

CREATE COLLECTION:

db.createCollection("students")

INSERT ONE:

db.students.insertOne({...})

INSERT MANY:

db.students.insertMany([{...}, {...}])

READ ALL:

db.students.find()

READ ONE:

db.students.findOne({...})

FILTER:

db.students.find({ marks: { $gt: 80 } })

AND:

db.students.find({ city: "Nashik", marks: { $gt: 80 } })

OR:

db.students.find({ $or: [{city:"Nashik"}, {city:"Pune"}] })

IN:

db.students.find({ city: { $in: ["Nashik", "Pune"] } })

UPDATE ONE:

db.students.updateOne({...}, {$set:{...}})

UPDATE MANY:

db.students.updateMany({...}, {$set:{...}})

INCREMENT:

db.students.updateOne({...}, {$inc:{marks:5}})

DELETE ONE:

db.students.deleteOne({...})

DELETE MANY:

db.students.deleteMany({...})

SORT ASCENDING:

db.students.find().sort({marks:1})

SORT DESCENDING:

db.students.find().sort({marks:-1})

LIMIT:

db.students.find().limit(3)

SKIP:

db.students.find().skip(2)

COUNT:

db.students.countDocuments()

DROP COLLECTION:

db.students.drop()

DROP DATABASE:

db.dropDatabase()

==========

IMPORTANT MONGODB OPERATORS

==========

Comparison:

$eq = equal

$ne = not equal

$gt = greater than

$gte = greater than or equal

$lt = less than

$lte = less than or equal

$in = matches any value in an array

$nin = does not match values in an array

Logical:

$and

$or

$nor

$not

Update:

$set

$unset

$inc

$rename

$push

$pull

Query / Element:

$exists

Sort:

1 = ascending

-1 = descending